

Guide to Key Architecture and Design Diagrams for a DDD-Based Authentication Service

Overview: This guide explains how to create and interpret various architecture and design diagrams for a modular authentication service built with Domain-Driven Design (DDD) principles, .NET 10, Dapper (for data access), and JWT-based security. Using a sample domain model (including `User`, `Role`, `Permission`, `Group`, `Session`, `Token`, `AuditLog`, etc.), each section covers the **purpose** of the diagram, its **structure and elements**, **how to build it using Draw.io**, and tips on **interpreting** the finished diagram. The diagrams span the full end-to-end architecture – from frontend/UI and API interactions, through domain logic and data persistence, to the deployment setup.

1. Use Case Diagram

Purpose

Use Case Diagrams illustrate the functional requirements of the system and how external actors interact with it. They provide a **high-level view** of what the authentication service does (or must support) without delving into implementation details ¹. In the context of an auth service, a use case diagram helps identify **user goals** (e.g. “Log In”, “Register Account”) and **admin goals** (e.g. “Manage Roles”) that the system must fulfill. This diagram is typically created during the requirements phase to ensure all stakeholder interactions and scenarios are captured ². It helps communicate with non-developers by focusing on *what* the system does rather than *how* it does it.

Structure

A use case diagram is composed of a few core elements: - **Actors:** External users or systems that interact with the authentication service, drawn as stick-figure icons. For example, a “User” actor represents an end-user of the system (logging in, etc.), and an “Admin” actor might represent an administrator managing accounts. - **Use Cases:** Oval shapes naming each significant functionality or goal the actor achieves with the system ³. Each use case represents a specific **scenario** or **required feature** (e.g. *Authenticate User*, *Reset Password*, *Issue JWT Token*, *Assign Role to User*). - **System Boundary:** (Optional) A rectangle enclosing the use case ovals, labeled with the system name (e.g., “Auth Service”). This boundary distinguishes which use cases are part of our system versus external processes. - **Relationships:** Lines connecting actors to the use cases they participate in. Additionally, *include* and *extend* relationships (dotted arrows with `<<include>>` or `<<extend>>` labels) can show dependencies between use cases ⁴. For instance, “Authenticate User” might **include** “Validate Credentials” (a required sub-step), or an Admin’s “Deactivate User” use case might **extend** a basic “Update User” use case (as a special scenario).

In our authentication domain, typical use cases would include: **User Login**, **User Logout**, **User Registration**, **Request Password Reset**, **Reset Password**, **Validate Token** (by other services), **Manage Users** (admin), **Manage Roles/Permissions** (admin). The *User* actor would be linked to personal actions like login or token refresh, whereas an *Admin* actor links to management use cases. By laying out these actors

and goals, we ensure the system supports all needed interactions (for example, that an Admin can assign a Role to a User, or a User can retrieve a new access token using a refresh token).

How to Build in Draw.io

1. **Enable UML Shapes:** In Draw.io (diagrams.net), open **More Shapes** and under *Software*, check the “UML” shape library, then click *Apply*. This loads common UML icons ⁵ including stick figures and ovals.
2. **Drag Actors and Use Cases:** Use the *Actor* shape (stick figure) for each external role (e.g., User, Admin). Use the *Use Case* oval shape for each identified functionality. Name the ovals with concise action phrases (e.g., “Log In”).
3. **Draw Connections:** Connect actors to their use cases with straight lines. In Draw.io, you can drag a line from an actor to an oval. Label any special relationships: if one use case includes another, draw a dotted arrow from the base use case to the included one and add `<<include>>` next to it ⁴. Similarly use `<<extend>>` for optional extensions. For example, you might have a “Authenticate User” use case that *includes* “Validate Credentials” and *extends* “Audit Login Attempt” (to log the event).
4. **Organize and Annotate:** Enclose the use cases in a rectangle to represent the system boundary (use a container shape or a rectangle from General shapes). Label it “Authentication Service” to clarify scope. Ensure the diagram remains uncluttered – group related use cases or use color-coding if needed for clarity.

Interpretation

To interpret a use case diagram, identify each actor and their associated use cases at a glance. **Actors** (stick figures) at the diagram’s edges indicate who or what interacts with the system. Solid association lines link actors to the **use cases** (ovals) they can execute ³. For example, if a line connects the *User* actor to the *Log In* oval, it means logging in is a function the user initiates. If an *Admin* actor is connected to *Manage Roles*, the admin can perform role management. Relationships like `<<include>>` show that one use case is **always called** as part of another (e.g., *Validate Credentials* is required whenever *Log In* happens), whereas `<<extend>>` indicates an optional or conditional scenario extending a base use case (e.g., *Add MFA Challenge* might extend *Log In* if multi-factor authentication is enabled). The use case diagram does **not** show the sequence of steps or how the system internally performs these functions – it simply outlines *what* the system does and *who* triggers it ⁶. By reviewing this diagram, stakeholders can verify that all needed authentication scenarios are accounted for and see the system’s scope of functionality at a high level.

2. Class Diagram

Purpose

A class diagram is a **structural diagram** that models the static design of the system – the classes (or types of objects) in the software and the relationships between them ⁷. In a Domain-Driven Design-based auth service, the class diagram serves to visualize the **domain model**: the entities, value objects, and possibly domain services that encapsulate the authentication business logic. This diagram helps the team understand the data structures and their associations (like which classes know about or contain others) before coding ⁸. It’s particularly useful for ensuring the design reflects the domain’s invariants and relationships (for example, confirming that a `User` has a collection of `Role` assignments, or that a

`Session` is linked to a single `User`). In summary, the class diagram is the blueprint of the software's *static architecture*, showing what objects will exist and how they relate.

Structure

Each **class** is drawn as a rectangle divided into compartments: the top compartment holds the class name, the middle lists attributes (fields/properties), and the bottom lists operations (methods) ⁹. In UML, you may include visibility markers (`+` public, `-` private, `#` protected) and data types for clarity, though high-level design diagrams sometimes omit method details for brevity. Key structural elements and notations include:

- **Classes (Entities/Aggregates):** In our auth domain, classes like `User`, `Role`, `Permission`, `Group`, `Session`, `AuditLog` are *Entities* (they have an identity and lifecycle). Some entities serve as **Aggregate Roots** – a design pattern where one entity (root) controls a cluster of related objects for data consistency ¹⁰. For example, `User` could be an aggregate root that encompasses related value objects (like a Password or Email object) and child entities (like `Session` records or an `AuditLog` of user actions). We denote an aggregate root usually just by context (it's the central class to which others are related).
- **Value Objects:** These are classes that represent a descriptive aspect of the domain without a unique identity ¹¹ ¹². In an auth system, examples might include a `Password` value object (encapsulating password hash and salt with validation logic) or a `JWT Token` object representing a signed token's value and expiry. In the class diagram, value objects are often shown as classes too, but understood to be used by entities (e.g., a `User` class might have an attribute of type `Password` or `Token` rather than a primitive string, indicating those are value object types).
- **Associations:** Lines connecting classes indicate relationships. A plain line means a relationship/knowning about the other class (solid line with no arrow implies a bidirectional association, or you can add an arrowhead for unidirectional) ¹³. For instance, `User` — `Role` could be a many-to-many association: one user can have multiple roles and each role is assigned to multiple users. This could be drawn as two classes connected by a line, with multiplicity *markers* near each end: `User * --- * Role` (or `User 0..* --- 0..* Role`) to denote many-to-many. A one-to-many would be, e.g., `User 1 --- * Session` (one user, many sessions).
- **Aggregation and Composition:** These are stronger forms of association. **Aggregation** (hollow diamond at the container end) means one class is a collection of others, but the parts can exist independently ¹⁴. **Composition** (filled diamond) means a strong ownership; if the container is destroyed, the parts are as well ¹⁵. For example, if we consider `User` and `Session`: a `Session` might be considered part of a `User` (especially if sessions are managed in-memory), which could be a composition (`User` composed of `Sessions`). But if sessions are stored globally with user IDs, we might just show an association with multiplicity. Use aggregation/composition notation when it clarifies design intent (like `Group` aggregates `User` references, or `Role` aggregates `Permission` list).
- **Generalization (Inheritance):** A solid line with a hollow triangle arrow pointing to the parent class denotes inheritance. In the auth domain, we might not have a lot of subclasses, but for instance, if `AdminUser` is a special type of `User`, it could inherit from `User` (though roles could also cover that, so it's just a theoretical example). Another example: if you had a base class `AuditLogEntry` and sub-classes for different event types, you'd use inheritance.
- **Multiplicity and Other Notations:** As mentioned, near the ends of association lines, denote cardinalities like `1`, `0..1`, `*` (many), or specific ranges (e.g. `1..*`) to convey how many instances can relate ¹⁶. For instance, next to `User` on the `Session` relationship, put `1` and next to `Session` put `0..*` (a user can have zero or many sessions active). These notations help in interpreting constraints in the domain model.

Applied to our example domain model: - `User` class might have attributes like *UserId*, *Name*, *Email*, and operations like *Authenticate()* or *ChangePassword()*. It may have an association to `Role` via a collection (e.g., a list of roles or role IDs) – showing that relationship in the diagram is essential to reflect authorization logic

(a user's roles determine permissions). - **Role** class could have *RoleId*, *Name*, and a collection of **Permission** (assuming roles bundle permissions). If roles are treated as an aggregate root, they might contain **Permission** value objects or entities. Permissions could also be modelled as separate entity nodes connected via an association (many-to-many if each permission can belong to multiple roles). - **Group** class might represent grouping of users (with attributes *GroupId*, *GroupName*). A **Group** may have an association to **User** (a group contains many users; a user can belong to many groups – again a many-to-many). We might also depict that a Group can have roles (if, say, roles can be assigned at group level). - **Session** class could include *SessionId*, *UserId* (FK), *RefreshToken*, *ExpiresAt*. In the diagram, **Session** is associated with **User** (each session belongs to exactly one user). Possibly marked composition if sessions are tightly owned by the user. **Token** might be a value object class containing the JWT string and maybe issued time/expiry – used as an attribute in **Session** or in **User** (for an *AccessToken* that isn't stored but generated). - **AuditLog** class might log events like *AuditLogId*, *UserId*, *Action*, *Timestamp*. It would have a relationship to **User** (many audit entries per user).

By laying out these classes and linking them, the class diagram reveals the **domain structure**: e.g., you might see that **User** is central, connected to Role, Group, Session, AuditLog – conveying that users tie to those aspects. This static view is vital for implementing the DDD model correctly.

How to Build in Draw.io

1. **Enable UML Shapes:** As with use cases, go to **More Shapes** in Draw.io, and enable “UML” (and optionally “UML 2.5”) shape libraries ¹⁷. This gives you pre-formatted class boxes and various relationship connectors.
2. **Create Class Boxes:** Drag out a *Class* shape for each primary class in the domain. Double-click to edit the compartments. For a high-level diagram, you might list just the class name and a few key attributes (e.g., **User** with attributes like **+Id**, **+Email**, etc.) for clarity ⁹. If focusing on design, you can omit method details or include one or two important operations (like **+Authenticate()** on **User**).
3. **Add Associations:** Use the connector lines to link classes. In Draw.io's UML library, you'll find different line endings for association, aggregation, composition, inheritance, etc. For example, select a line with a diamond end for aggregation and drag it from the container class to the contained class (e.g., from **Role** to **Permission** if *Role aggregates* permissions). For simple associations, a solid line is fine (you can add an arrowhead by editing the line style to indicate direction if needed). Label the associations with role names or cardinalities by double-clicking the line and typing (e.g., near the **User** end write “1”, near the **Session** end write “0..*”).
4. **Use Inheritance and Interfaces if Applicable:** If you have a generalization (inheritance), use the line with a triangular arrowhead. For example, if you have an **IPermissionAssignable** interface implemented by both **User** and **Group** (just as an example of an interface indicating they can have Roles), draw a dashed line with hollow triangle from the class to the interface (UML realization). In Draw.io, the UML shape library will have interface shapes (often depicted similar to classes or as lollipops in component diagrams; for class diagram, you can use a class shape with <<interface>> stereotype). Position child classes below parent classes and connect upward with the inheritance arrow.
5. **Refine Layout:** Align classes in a logical way – e.g., you might place the main aggregates (**User**, **Role**, **Group**) centrally, with value objects off to the side attached to them, and less central entities like **AuditLog** or **Session** below or aside the **User**. This visual grouping helps readers see clusters (maybe draw a dashed box around the *Identity and Access Context* classes to indicate the bounded context, if

desired). Ensure no lines overlap confusingly; you can use elbow (angled) connectors to route around boxes.

Interpretation

Reading a class diagram involves understanding the **domain structure and constraints** it visualizes: - First, identify the primary classes (often those with many connections). In our case, `User` would likely be a focal class – it may be connected to `Role`, `Group`, `Session`, etc. This suggests `User` is an important aggregate or concept. If the diagram has a label like `<<AggregateRoot>>` on `User` or `Role`, that tells you those classes govern an aggregate (this can be optionally noted as a stereotype). - Look at the **associations** and their multiplicities to infer relationships. For example, if you see `User 1 --- * Role`, you know each user can have multiple roles, and if there's a corresponding `Role * --- * User`, it confirms a many-to-many (likely implemented via a linking table or collection in code). A composition diamond from `User` to `Session` (`User ◆— Session`) with `1..*` on `Session`'s end implies that sessions belong strictly to a user and cannot exist beyond it (e.g., deleting a user would remove their sessions). Aggregation or association without a filled diamond (`User ◇— AuditLog`) might indicate a weaker relationship (audit logs exist independently but reference the user). - Note any **inheritance** relationships. If, say, `Admin` inherits `User`, an open triangle arrow from `Admin` to `User` shows that `Admin` is a specialized `User`. This tells the reader that wherever a `User` is expected, an `Admin` can be used (Liskov substitution), and `Admin` likely adds extra properties. In DDD, such subclassing is less common than using roles or value objects, but the diagram will make it apparent if present. - **Value objects** or other compositions often appear as classes with no standalone links apart from their containing class. For instance, if `Token` is drawn but only connected to `Session` or `User` with a composition, it signals `Token` is not used independently (probably a value object embedded in `Session`). The interpretation is that `Token`'s lifecycle is tied to that owner. We can also infer immutability or conceptual significance (since value objects “describe things”¹⁸, seeing one in the diagram highlights that concept in the model – e.g., a `Password` value object class indicates the domain treats password data with special rules). - The class diagram can also reveal **navigability** or dependency direction. For example, if the association arrow points from `Session` to `User` (and not vice versa), it means a `Session` knows which `User` it belongs to, but the `User` class might not explicitly keep a collection of sessions. That might hint at how the code is organized or how aggregate boundaries are set (maybe sessions are retrieved via a repository by user ID rather than through a user's in-memory collection). - Because this authentication service follows DDD, pay attention to **aggregates**: Only aggregate roots (like `User` or `Role`) should be referenced directly by other aggregates¹⁹. If the diagram shows that, for instance, no class other than `User` references `Session`, that implies `Session` is an internal part of the `User` aggregate. This adheres to the rule that aggregate roots guard their entities and outside objects only hold reference to the root²⁰. Such nuances might not be explicitly labeled, but the absence or presence of certain cross-links is telling. For example, if no line directly connects `Session` to `Group` or `Role`, it reinforces that `Session` is only relevant through `User`. - Finally, consider the **domain meaning** of the structure. A well-structured class diagram aligns with real-world concepts. If you see `User -> Role -> Permission`, you interpret that as: Users have roles, roles confer permissions (classic RBAC model). If you see `Group -> Role` too, that means groups also have roles (maybe to mass assign permissions to all group members). As a reader, you can then question or confirm: does the system require both user-specific and group-based role assignments? The diagram prompts such domain questions and shows the intended design answers.

By thoroughly examining the class diagram, one gains insight into how the **authentication service's domain logic is structured** in code – which classes exist for core concepts, how they relate, and how DDD principles like aggregates and value objects are applied.

3. Sequence Diagram

Purpose

A sequence diagram is a type of **interaction diagram** that details how different parts of the system interact over time to accomplish a specific **scenario or use case** ²¹. It shows the ordered sequence of messages (method calls, signals) between objects or components as the scenario unfolds. The main purpose is to capture the **flow of control** and data in a particular process – for an authentication service, common examples include the login process, token validation, or user registration flow. Whereas a use case diagram tells *what* happens abstractly, a sequence diagram shows *how it happens step-by-step* in the system, making it closer to actual code execution or workflow ²². It is especially useful to designers and developers for understanding how responsibilities are distributed among components (e.g., what does the controller handle vs. the domain service vs. the repository) and for verifying that the interactions meet the required behavior (e.g., ensuring the system checks credentials before issuing a token, etc.). Sequence diagrams also help in identifying potential integration points or error paths in a workflow (like what happens if credentials are invalid).

Structure

Sequence diagrams have a straightforward but rich structure:

- **Participants (Lifelines):** These are the entities involved in the interaction, drawn as labeled boxes or stick figures at the top, with a dashed line (lifeline) descending from each ²³. Participants can be objects, class instances, components, actors, etc. In our authentication scenario, participants might include: the *User* (or UI/client), the *Auth API Controller* (handling HTTP request), the *Authentication Service or UseCase* (application layer that orchestrates the login), the *Domain Model* objects (like a User domain object or a Domain Service for token generation), the *Repository* (using Dapper to query the database), and the *Database* (SQL Server). We might not explicitly show the DB as a lifeline if we treat repository calls as the end, but it can be included to illustrate the query.
- **Lifelines & Activation Bars:** Each participant has a lifeline (vertical dashed line) that represents its existence over time during the scenario ²³. When a participant is actively executing or processing a message, we often draw an **activation bar** (thin vertical rectangle on that lifeline) to show the time span of that execution ²⁴. For example, when the Auth Controller receives a login request, it has an activation bar while it calls the auth service and waits for a response.
- **Messages:** Horizontal arrows between lifelines denote messages or method calls. A solid arrow line indicates a call or request, optionally with a description (message name) above it ²⁴. A dashed arrow back indicates a return (with an optional return value or note) ²⁵. The sequence is read top-to-bottom; the vertical ordering of arrows shows the time order of interactions ²⁶. In our login example, messages could be: **User** -> **Auth API** ("POST /login with credentials"), **Auth API** -> **AuthService** ("Authenticate(username,pw)"), **AuthService** -> **UserRepository** ("GetUserByUsername" query via Dapper), **UserRepository** -> **Database** (this can be implicit or shown as another lifeline), then returns with user data, then **AuthService** might invoke a **User domain object** method to verify the password, then **AuthService** -> **TokenService** ("GenerateJWT for user"), and so on, culminating in a result being returned to the user. Each of these steps is a message arrow in the diagram, in order.
- **Alternatives / Conditions:** Sequence diagrams can include *conditional flows or loops* using frame elements. An **alt** frame (with a guard condition label) can show alternate paths. For instance, after verifying credentials, a condition check can be shown: if credentials are valid (guard "[valid]") then generate token; if not (the "else" part) return an error. Loops can be shown with a **loop** frame (with an iteration condition). In a simple auth sequence, we might not need a loop, but an alt for valid vs invalid login is useful.
- **Notes:** You

can attach textual notes (comments) to clarify steps or highlight rules. E.g., a note that “Password verification uses BCrypt hashing” attached to the verification call.

A concrete example (in text form) of a **login sequence** might go as follows: 1. **User** sends a Login request (with username/password) to **Auth API** (REST endpoint) ²⁷. 2. **Auth API** (Controller) calls the **Auth Application Service** to authenticate the user. 3. The **Auth Service** uses the **UserRepository** (Dapper) to fetch user data from **Database** ²⁸. 4. Repository returns the User data to Auth Service. Auth Service then asks the **User** domain entity (or a Domain Service) to validate the password (this could be an internal method call, sometimes not shown if staying high-level). 5. If credentials are valid ²⁹, Auth Service generates an **Access JWT Token** and a **Refresh Token** (perhaps via a Token Domain Service or library) ²⁹. 6. Auth Service stores the new refresh token in a **Session** record (persist via repository) or in memory, and returns the tokens to the **Auth API**, which then returns them to the **User** as response ³⁰. 7. If credentials are invalid, Auth Service returns an error (401 Unauthorized) to the API, and then to the User ²⁹. 8. (Optionally) The User then uses the received JWT to call other services – those services will validate the token’s signature and claims locally on each request ³¹ ³² (this could be another sequence diagram, but it’s an extension of the auth flow: token usage).

This sequence aligns with a typical JWT auth flow (as described in an example e-commerce scenario) – the advantage is that after step 5, the auth service is not called for each request; other services verify the token until it expires ³³.

How to Build in Draw.io

1. **Enable UML Shapes:** In Draw.io, ensure the “UML” or “UML 2.5” library is enabled (it contains shapes for sequence diagrams like lifelines, activation bars, etc.) ³⁴.
2. **Add Participants (Lifelines):** Drag out the *Lifeline* shape for each participant. You can use the rectangle head with a dashed line tail. Name each lifeline at the top (e.g., **User**, **AuthController**, **AuthService**, **Repository**, **DB**). If the user is external, you might use an Actor stick figure instead of a lifeline rectangle for the user to indicate an actor.
3. **Stretch Lifelines:** Pull the lifelines down to an equal length on the canvas to accommodate the number of interactions. Lifelines will run in parallel columns.
4. **Draw Messages:** Use the line connectors for messages. In the UML shape library, you’ll find specific connectors like *Synchronous Message*, *Asynchronous Message*, etc. A simple solid arrow is fine for calls. Drag from the sender lifeline to the receiver lifeline at the appropriate vertical position. Label the message by double-clicking the line and typing (e.g., “loginRequest(username,pw)” or “ValidateCredentials”). For returns, use a dashed line arrow back from the receiver to sender. You may label return values (e.g., “User data” or “JWT token”) on these dashed lines, though it’s optional.
5. **Activation Bars:** If desired, drag an *Activation* box shape onto a lifeline to indicate when that participant is active processing. In Draw.io, when you drop an activation on a lifeline, it should attach (the outline turns purple when correctly attached) ³⁵. Size the activation bars to span from the message reception to the point of response. For example, put an activation on AuthController during the call to AuthService and end it when AuthService returns.
6. **Optional Frames for Conditions/Loops:** If your sequence has branches (e.g., valid vs invalid login), use a *Frame* shape. Label the frame’s operator in the top-left (e.g., **alt** for alternative). Within the frame, in a corner, you can put text like “[Invalid credentials]” on one section and “[Valid]” on the other section. Draw.io may not have a specific alt fragment shape, but you can simulate it with a rectangle or by using the *combined fragment* shape if available. Alternatively, simply annotate with an

if note. Simpler diagrams might skip explicit UML alt boxes and instead just use notes or a decision pseudocode in text.

7. **Notes:** For clarity, attach a *Note* shape (from General shapes or UML notes) to any lifeline or message if you need to explain something (e.g., a note attached to the token generation message: "JWT contains userId, roles, exp timestamp"). Connect notes with a dashed line to the relevant lifeline or arrow.
8. **Layout:** Space out the lifelines so message arrows are not cluttered. Align arrow labels for readability (you can drag the text). Ensure the sequence of calls is strictly top-down to maintain chronological order.

Interpretation

When reading a sequence diagram, follow it **top to bottom, left to right**: - Across the top, note the *participants* in the scenario. This sets the context of "who" is involved. For our auth example, seeing `User -> AuthController -> AuthService -> Repository -> Database` in that order indicates a flow from an external user through the system's layers down to the database. Each lifeline's label tells you which component or object is handling that part of the process. - Progressing downward, follow the **arrows** in order. They indicate the interactions step by step ²⁶. For each arrow: - The arrow's origin is the caller and the arrowhead is the receiver of the message. The label on the arrow is the action or method invoked. For instance, an arrow labeled "Authenticate(cred)" from *AuthController* to *AuthService* tells us the controller calls an `Authenticate` operation on the service, passing credentials. The next arrow, say from *AuthService* to *Repository* labeled "GetUserByName(u)" shows the service queries the repository. - Indented or nested arrows (those that start later, while another lifeline's activation is ongoing) imply a **call hierarchy**. In the diagram, if *AuthService* is active and during that, it calls *Repository*, the repository call arrow will be slightly lower and to the right of the *AuthService* call arrow. This nesting conveys that *AuthService's authenticate method internally calls Repository*. The vertical ordering gives the exact sequence. - Notice any **activation bars** – a long bar indicates that object is doing some work for that duration. If the bar extends while multiple calls happen within it, that object is orchestrating those sub-calls. For example, an activation on *AuthService* encompassing calls to the repository and token service means *AuthService orchestrates those steps as part of handling the authenticate request*. - Check for **conditional flows**: If an `alt` or decision is present, interpret the guards. E.g., the diagram might show two returns from *AuthService*: one return arrow labeled "JWT token" under a "[valid]" condition, and another return arrow labeled "Error 401" under "[invalid]". This means depending on the validity check, the outcome differs. Understanding these forks is crucial: it shows how the system handles both success and failure in one diagram. - Look for **loops** or repeated calls: Sometimes depicted with a *loop* frame or just an arrow pointing back upwards. In an auth flow, maybe a loop isn't typical, but if it were, it might show something like checking multiple authentication factors or iterating through permission checks. If present, ensure you catch that it means repetition until a condition is met. - Use the context of the domain: Given this is an authentication service, certain calls have expected meanings: - A call to the database with a query suggests data retrieval (e.g., verifying user exists and checking hashed password). - A call to a token generator implies cryptographic signing of JWT. - If you see a message like "AuditLog.RecordLogin(user)" from *AuthService* to *AuditLog*, that indicates the service logs the event. These domain-specific interactions clue you into non-functional steps like auditing. - Understand the **end result**: Typically, the final message in the sequence returns to the initiator (the user or calling system). In our login example, the last arrow would likely be from Auth API back to User labeled "JWT token + refresh token" or an error. This signifies what the user receives. Everything above it are intermediate steps that led there. - By interpreting each participant's role and the order of messages, you get a clear picture of how the *use case is implemented*. For instance, the diagram might show that the *AuthController* does very little except delegate to *AuthService*, which does a lot (checking DB, verifying password, creating

tokens). This tells the reader the design is keeping controllers thin and logic in the service/domain – a good practice. If the sequence had the controller directly talking to the DB, that would indicate a different design (maybe less layered). - Also, note any **timing or performance implications** that might be implicit. If the sequence shows many back-and-forth calls (like multiple queries), one could deduce potential latency or complexity. In an auth scenario, fewer calls (just one DB fetch and one token creation) is typical, which the diagram would confirm. - In summary, the sequence diagram should read like a story of the login transaction: *The user sends credentials, the system verifies them against the database, issues tokens, and responds* ³⁶ ³². By following along, one can verify the design handles all necessary steps in order (e.g., ensure that *validate password* happens **before** token issuance, or that *audit log* happens after successful login, etc.).

By carefully reading a sequence diagram, developers and stakeholders can validate that the system's runtime behavior will fulfill the use case as expected, and they can spot any missing interactions or unnecessary ones. It essentially answers, "How do we get from point A to point B in this scenario?" in a visual, traceable way.

4. Component Diagram

Purpose

A component diagram offers a high-level **structural view of the software system's components and how they interface with each other** ³⁷. It is useful for visualizing the architecture in terms of **modular parts** of the system (often corresponding to deployable units, libraries, microservices, or subsystems) and the relationships (especially interfaces and dependencies) between those parts. For a DDD-based authentication service, the component diagram can depict how the service is partitioned into layers or modules (for example, API layer, Application layer, Domain layer, Infrastructure) and how it interacts with external components (like a UI or other services) and infrastructure elements (like the database). It captures implementation details at a level higher than classes – focusing on **encapsulation and communication**. This is particularly helpful to ensure that the system's design is **modular, reusable, and follows separation of concerns** (e.g., showing that the JWT generation functionality might be in a separate component or library, or that the Dapper-based repository is part of an infrastructure component). Stakeholders can use this diagram to understand the **deployment units** and service boundaries without drowning in class-level detail.

Structure

Key elements in a component diagram include: - **Components:** Represented by specialized shapes (typically a rectangle with two smaller rectangles protruding on the side, or the stereotype `<<component>>` above the name) ³⁸. Each component is a logical unit of the system – for instance, *Authentication API*, *Auth Domain Service*, *Identity Repository*, *JWT Issuer Service*, etc., could be components in our system. A component can also represent an external system or library (like a *SQL Server Database* can be shown as a component with stereotype `<<database>>` ³⁹, or a *Logging Service* as an external component). - **Interfaces (Provided and Required):** Interfaces define the contracts for interaction. In UML component diagrams, provided interfaces (what a component offers) are shown as a "lollipop" symbol (a small circle) attached to the component border ⁴⁰, and required interfaces (what a component needs) are shown as an open socket (half-circle) ⁴⁰. You can also just use dependency arrows and textual notes for interfaces if the diagram is kept simpler. For example, our *Auth API Component* might provide an interface `IAuthService` that other

applications can call (HTTP/REST interface), and it might require an interface `IUserRepository` that is provided by the *Data Access component*. - **Ports:** A port (small square on the component boundary) can be used to indicate an interaction point for a component, especially if the internal structure is complex or to group interfaces ⁴¹. This might be overkill for a simple diagram, but for example, a port could show that the Auth Service component exposes a port for HTTP requests. - **Connections/Dependencies:** Lines or arrows between components show how components communicate. A straight line from one component's required interface (socket) to another's provided interface (lollipop) means the first component calls the second via that interface ⁴². In simpler terms, if using just boxes, you might draw a dashed arrow labeled "<<uses>>" from Auth Service to Database to indicate the dependency (or a solid line if showing association). Arrows can be omitted if using the lollipop notation since the connection is implicit in linking a socket to a lollipop. - **Stereotypes and Labels:** Components can be stereotyped to clarify their nature (e.g., <<UI>> for a user interface module, <<application>> for a running service, <<library>> for a shared library, <<database>> for the DB) ⁴³. This helps quickly distinguish their roles. In our scenario: the front-end could be a <<UI>> component, the auth service a <<microservice>> or <<application>>, the database as <<database>>, maybe Dapper as a <<library>> used by the repository component.

For the authentication service, a possible component breakdown is: - **Client Application (Front-end)** – not part of the auth service deployment, but an external component that *consumes* the auth API (e.g., a web app or mobile app). - **AuthService API** – the web/API component (e.g., an ASP.NET Core Web API hosting authentication controllers). It provides endpoints like `/login`, `/register`, etc. This component requires the Auth Domain's functionality. - **Auth Domain & Application** – a component implementing the core authentication logic (could be combined or split in diagrams; for example, a component for "Auth Domain" encapsulating domain model and domain services, and perhaps an "Application Layer" component orchestrating use cases). This component provides interfaces like `AuthenticateUser`, `ManageUserAccounts` that the API layer calls. It likely requires a Data Access interface. - **Infrastructure/Data Access (Repository)** – a component handling database access via Dapper. It provides something like `IUserRepository` (and others like `IRoleRepository`) which the domain or application layer uses. Internally it uses Dapper to run SQL on the DB. - **JWT Provider (Security Component)** – optionally, we could show a component (or external service) that handles JWT creation/validation. In .NET, JWT generation might just be a NuGet library, which can be represented as a <<library>> component or omitted. But if treating it logically, we could have a `TokenService` component that provides token creation. - **Database** – shown as a component (often drawn as a cylinder icon or a component with stereotype <<database>>). The Repository component connects to this.

The relationships would be: UI calls AuthService API (provided interface like a REST API), AuthService API uses Auth Domain's interface (or they could be grouped as one component depending on detail level), Auth Domain uses Repository's interface to fetch/store data, Repository in turn connects to Database.

How to Build in Draw.io

1. **Enable UML shape library:** Ensure the "UML" and/or "UML 2.5" libraries are on ⁴⁴. Look for *Component* shapes – Draw.io provides a few styles (some with the component "chip" icon in the corner) ⁴⁵.
2. **Drag Component Shapes:** For each identified component in your architecture, drag a component shape onto the canvas. Name them clearly (e.g., *Auth API (Web)*, *Auth Application/Domain*, *Auth Repository*, *SQL Server DB*, *Client App*). You can use the stereotype notation by editing the text to

include `<< >>` if desired (e.g., `<<component>> Auth API` or `Auth API <<application>>`). Some shapes might already display the `<<component>>` keyword icon by default.

3. **Add Interfaces (if using explicit):** If you want to depict provided/required interfaces, find interface circle (lollipop) shapes. For a provided interface: drag a small circle, attach it to the component's edge (Draw.io might not snap automatically; you may group them visually). Label the interface (e.g., `IAuthService`). For a required interface: draw a small half-circle (socket) symbol or you can simulate by using a circle and an open circle together. Alternatively, skip the symbols and simply use connecting lines with text labels indicating the interface/protocol (like an arrow labeled "HTTP/REST" from Client to Auth API).
4. **Connect Components:** Draw connections between components. If using lollipop/socket, connect the socket line to the lollipop circle ⁴⁰. In Draw.io, you might use a regular line but visually position it so the socket and lollipop appear linked. If not using those symbols, just draw a line from one component to another. Use dashed lines for dependency or solid lines for association (UML recommends a dependency arrow for "uses" relationships). For example, draw a dashed arrow from `AuthService` component to `Repository` component and label it `calls` or the interface name (`IUserRepo`). From `Repository` to `Database`, you could draw a solid line labeled "SQL connection" or an interface like `IDatabase`.
5. **Use Note or Stereotype Labels:** You can add small text by each connection to indicate what protocol or interface is used (e.g., "HTTP", "SQL", "JWT" etc.). Optionally, group components into higher-level packages if needed (e.g., draw a large container box around all Auth service internal components to distinguish from external client and database).
6. **Layout:** Organize the components in a logical flow. Often, diagrams are drawn left-to-right or top-to-bottom in the direction of data flow. For instance, place the *Client* on the left, pointing to *Auth API* next to it, then *Auth Domain* next, then *Database* at far right. Or stack them with Client at top, Auth Service in middle, DB at bottom, and draw arrows downward. Choose an arrangement that avoids crossing lines.
7. **Iconography:** Draw.io has some network shapes; you could use a cylinder shape for the DB if preferred (though UML component with `<<database>>` text is fine). Similarly, an actor figure or a browser icon can represent the client. This is acceptable as component diagrams often mix in illustrative icons for clarity (just ensure to label them).

Interpretation

When you examine a component diagram, you're looking at the **big building blocks** of the system and how they interact:

- Identify each box (component) and read its name and stereotype. This tells you *what* each part is. For example, seeing a component labeled "AuthService API `<<component>>`" and another "AuthRepository `<<database>>`" immediately signals one is a web service part and another is related to data persistence ⁴⁶. If the client application or other external systems are shown, you know what initiates communication.
- The **connections/interfaces** between components indicate *who talks to whom and how*. If there are lollipop and socket symbols, a lollipop attached to Auth API means "Auth API offers this interface to others" – if a line from the Client's socket connects to that lollipop, you interpret that as the client uses the Auth API's interface (for example, the client calls the REST API provided by the AuthService). A dependency arrow from AuthService to Repository implies AuthService relies on the repository component's functionality (likely via some interface contract like a repository interface), meaning if you replace that repository, AuthService would be affected.
- **Directionality matters:** an arrow or lollipop linkage going one way means one component depends on or calls the other, not vice versa (unless there are two arrows or a double-headed arrow showing two-way communication). For instance, the diagram might show AuthService -> Database (meaning AuthService writes/reads the database, but the database doesn't call AuthService). If

an arrow is labeled “uses JWT library”, from Auth Domain to JWT component, that means the domain uses an external JWT component. - Look at **groupings or layers** implied: In a DDD system, typically you’ll see groupings like presentation (API), domain, infrastructure (DB). The diagram might explicitly reflect layers: e.g., the Auth API and Auth Domain could be shown as separate components or combined (depending on how the system is deployed). If separate, an arrow from API to Domain shows a strict layering (API calls into domain logic). If combined, the internal separation might not show here, instead one component “AuthService” encapsulates both but maybe with notes or internal interfaces. - Check if external dependencies are properly delineated. For example, the database being a separate component indicates that the system treats it as an external element (which it is). If the diagram shows the database with a provided interface (like SQL interface) and the repository requiring it, that underscores the separation of concerns – the repository component can be thought of as the adaptor between domain and the database’s interface. - **Interface names or protocols**: They clarify how components connect. If you see something like `IAuthService` or “REST API” between Client and Auth API, you understand the client communicates via HTTP calls. If between AuthService and Repository you see `IUserRepository` interface, that means the AuthService expects that contract for data operations. If between Repository and Database it says “ADO.NET/Dapper” or just an arrow with no interface (since databases typically use query languages), you infer it’s direct DB access. - **Modularity and Reuse**: A good component diagram shows components that could be replaced or scaled independently. For example, the JWT Provider could be an internal class library or an external service; seeing it as a component implies it could potentially be swapped (maybe using a different JWT library or an auth server) without affecting other components as long as it fulfills the same interface. If an **Admin UI** or an **API Gateway** component was shown interacting with AuthService, that indicates how this auth service fits in a larger ecosystem (e.g., being called by other system parts). - Note also any **packaging or deployment cues**: Sometimes, component diagrams double as a precursor to deployment diagrams by indicating which components might reside in the same process or deployment unit. For instance, if the diagram clusters Auth API and Auth Domain inside a higher-level “AuthService” box, that implies they are deployed together (as one service). If they’re separate boxes, maybe they are separate processes or modules (though UML component doesn’t strictly define deployment – that’s for deployment diagrams, covered later). - Overall, by reading the component diagram, you should grasp the **architecture’s modular structure**: For example, “Clients call the Authentication component via HTTP; the Authentication component uses a Repository component to talk to a SQL Database; there’s also a Token generation component/library involved.” Each component’s role is clear and the connections show the allowed communication paths ⁴⁷ ⁴⁸. This is crucial in a DDD system to maintain boundaries (the component diagram can enforce that, say, the UI never talks to the database directly, it must go through the service’s interface, etc.).

In essence, the component diagram provides a map of **the codebase or system at a high level** – indicating what pieces exist and how they cooperate. It helps ensure that the system is cleanly separated (e.g., that domain logic is not directly mixed with UI or DB code) and that all interactions between parts are well-defined (often using interfaces), thus supporting a modular, maintainable design.

5. Deployment Diagram

Purpose

A deployment diagram captures the **physical deployment architecture** of the system – it shows the runtime environment, including hardware nodes, servers, containers, or cloud infrastructure, and how software components are deployed onto this hardware ⁴⁹. The focus is on **where** each part of the system runs and how different computing nodes communicate. For our .NET 10 authentication service, a

deployment diagram would illustrate how the service is hosted (e.g., on a web server or cloud app service), where the database resides (e.g., a separate DB server), and any other relevant nodes (like a client machine, load balancer, etc.). This diagram is important for understanding the system's topology, network relationships, and for planning capacity and DevOps considerations. It helps answer questions like: Are the web server and database on the same machine? Is there a separate server for the front-end? How do requests travel through the network? In an enterprise setting, it can also show firewalls, DMZs, or microservice containers if applicable. Essentially, it's the blueprint for deployment in production or other environments, highlighting hardware/software mapping ⁵⁰.

Structure

Deployment diagrams consist of:

- **Nodes:** These are the physical or virtual machines, servers, devices, or execution environments. In UML, nodes are typically drawn as 3D cubes or labeled boxes, sometimes with a device icon. A node can represent a physical server, a cloud instance, a user device, etc. ⁵¹. You can also have nodes that represent execution environments (e.g., a Docker container or a JVM, or an IIS process) inside a physical node. Nodes can nest to show one is deployed on another (like an application running on a server node).
- **Artifacts:** Inside nodes, you can list the software artifacts deployed there (e.g., executables, DLLs, database files). Artifacts are usually shown as rectangles with folded corner (document icon) and can be stereotyped (like `<<executable>> AuthService.dll`). In many cases, people simplify by just naming the component or system on that node.
- **Connections:** Nodes are connected by communication paths (represented by lines, often with a stereotype indicating protocol or network type). For example, a line between the Web Server node and Database Server node could be labeled `JDBC` or `SQL TCP/IP`, or simply left as an unlabeled line (implying network connection). If internet/cloud is involved, sometimes a cloud symbol or a note like "HTTPS" is added to the connection.
- **Stereotypes/Device Types:** We often mark nodes with stereotypes like `<<Server>>`, `<<Database Server>>`, `<<Browser>>`, `<<Mobile>>` etc., to clarify what each node is. For example, a client node might be `<<Client PC>>` or `<<Mobile Device>>` where the front-end runs. The database node might be `<<DB Server>>`.
- **Environment/Zone grouping:** Optionally, you might group nodes by network zones or environments (for instance, putting the web server and DB server inside a "Corporate Network" box and the user outside it), or using node grouping to indicate clusters (like multiple web server nodes behind a load balancer).
- **Execution Environment Nodes:** UML allows showing an environment like ".NET runtime" or "Kubernetes Pod" as a node that contains the component artifact. For a .NET 10 service, you might show a node `<<Docker Container>> AuthService Container` inside a `<<VM>>` node if containerized, for instance. If using Azure, you might label a node `<<Azure App Service>> Auth Web App`.

For our authentication service, a typical deployment scenario might be:

- A **Client Node** (not part of deployment per se, but to show the user's side). This could be a user's browser or device from which requests originate. You can denote it as a node or an actor icon. It's outside the server infrastructure.
- A **Web Server Node** (or cluster of nodes) that hosts the *Auth API*. If it's a cloud deployment, this could be an *App Service* or *Kubernetes Pod* running the .NET 10 application. We might label it something like "AuthService Server (Ubuntu VM) `<<Node>>`" and inside it, list an artifact like `AuthService.dll (ASP.NET Core App)`.
- A **Database Server Node** that hosts the *SQL Server database*. For example, a node labeled "SQL Server DB Node" with an artifact "AuthDB.mdf" or just mention the schema name. If using cloud DB, maybe `<<Azure SQL>> AuthDB`.
- Possibly an **API Gateway or Load Balancer** node if the auth service is behind one. Also, if using a separate identity provider or caching server, those nodes could be included. But given just an auth service example, maybe not needed.
- **Connections:** The diagram would show the client node connecting to the web server node over the internet

(perhaps labeled “HTTPS”), and the web server node connecting to the database server node (labeled “SQL/TCP” or just an unlabeled line meaning network call to DB).

How to Build in Draw.io

1. **Enable UML or Networking Shapes:** The UML library in Draw.io has node and deployment shapes, but you can also use general shapes (the cube shape is common for nodes). Draw.io specifically offers a “Deployment” shape in the UML 2.5 library, which is a cube-like shape ⁵². Enable “UML” and “UML 2.5” shapes ⁵³. You may also enable “Network” shapes for routers/cloud if needed.
2. **Add Nodes:** Drag out the *Node* shape (which might appear as a cube or as a labeled box). Label each with a name and stereotype. For example: a node labeled `Web Server <<Node>>` or `<<Application Server>> AuthService Host`. Another node labeled `<<Database Server>> SQLServer`. For the client, you could drag a *Device* shape (like a person or a computer icon) or just another node labeled `User Browser`.
3. **Nest Artifacts (or Components):** Inside the web server node shape, you can place a smaller component or artifact shape representing the deployed software. For instance, drag a folded-corner *Artifact* shape into the Web Server node and label it `AuthService.exe` or `AuthService (Docker Container)` depending on how specific you want to be. Similarly, inside the DB node, put an artifact like `AuthDB (SQL Database)`. If Draw.io doesn’t allow drawing inside, you can simulate by placing shapes on top and grouping. Alternatively, simply annotate the node text to include deployed items (e.g., “AuthService API (Docker container running .NET 10 app)” as the node label).
4. **Connect Nodes:** Use solid lines to connect the nodes, symbolizing network connections. If needed, add a lightning bolt icon or similar for network. Label these connections with protocols or port info if relevant: e.g., “HTTPS 443” between client and web server, “SQL 1433” between app server and DB. You can use a text label on the line or a `<<stereotype>>` on the line (like `<<HTTP>>`).
5. **Multiple Instances or Clusters:** If you have more than one instance (like two web servers behind a load balancer), you can draw multiple node cubes labeled “AuthServer1” and “AuthServer2” or stack them, and then a load balancer node in front. Indicate that the client talks to LB, which distributes to servers. If not needed, one server is fine.
6. **Environment Annotations:** You can use a dashed box to group nodes (for example, a box labeled “Cloud VPC” containing both the app and DB nodes to show they are in the same network). Or label outside connections with a “firewall” note if applicable.
7. **Styling:** Optionally, use icons for clarity: Draw.io has some Azure and AWS icon libraries if you want a fancy depiction (like the Azure SQL database icon). But a simple UML generic approach is fine. Ensure nodes are clearly separated visually.
8. **Review for Completeness:** Check that every major **physical element** is included. For instance, if the auth service uses an external Identity provider (like AWS Cognito or something) that should appear if in scope. If not, just keep to the direct pieces.

Interpretation

Reading a deployment diagram is about understanding the **physical layout** of the system: - Identify each **node** and what it represents. The stereotype or icon will hint at this: e.g., a node with `<<Server>>` or that looks like a cube likely stands for a server machine or VM. If the label says “AuthService Web Server (Linux VM)”, you know the auth application runs on a Linux VM. If another node is “SQL DB Server”, that’s the database host. A user’s device might be depicted with a person or laptop icon, clarifying the client side. -

See what **software artifacts** live on each node. For example, on the Web Server node, you might see an artifact named "AuthService.dll" – indicating the .NET Core application is deployed there. On the Database node, an artifact "AuthDB" indicates the database schema or instance is hosted on that node. If artifacts aren't explicitly drawn but the node is named clearly (like "Auth Service (Docker Container)" as the node itself), then you infer that's the software running there. - Look at the **network connections** between nodes. The lines connecting nodes show which nodes communicate. For instance, a line between User's browser and the Auth Web Server means the user can send requests to that server. If labeled "HTTPS", it tells you it's over an encrypted HTTP connection. A line between the Auth Server and the DB server likely means the application queries the database over the network (likely TCP connection for SQL). If multiple web servers connect to one DB, you'll see multiple lines converging on the DB node, indicating all instances share that database. - Consider any **performance or availability implications**: For instance, if the diagram shows multiple app servers behind a load balancer, you interpret that as the system is scaled out for the web tier. The presence of a load balancer node (often drawn as a funnel or something) indicates clients don't directly talk to servers but via that LB. If only one of each, then it's a simpler deployment (but also a potential single point of failure). - Note **physical locations or boundaries** if given: Sometimes diagrams differentiate between on-premise vs cloud by grouping nodes or notes. E.g., if the Auth server and DB are in a cloud network and the user is outside, you might see a demarcation like "Internet" between the user and cloud. Interpreting that, you understand which communications traverse the internet (user to service) and which are internal (service to DB likely in same secure network). - Security elements: If shown, a firewall icon or a demilitarized zone (DMZ) might be present between user and server, meaning the server might be in a protected subnet. Interpreting that, you'd know the network is segmented. - Summarizing the scenario: For our auth service, one could narrate the deployment diagram like: *"The authentication service is deployed as a container on an Azure App Service (Web Server node) which connects over the network to an Azure SQL Database (DB node). Users from their browsers/mobile apps send HTTPS requests to the App Service's public endpoint. The App Service and the Database reside in the cloud environment. The system's deployment consists of these two main nodes plus the client side."* Each element's role in deployment is clear: the web node runs the .NET app, the DB node stores persistent data, and the user interacts via the network ⁵¹. - Compare to the component diagram: The deployment diagram should reflect where the components live. For example, if the component diagram had an AuthService and a Database component, the deployment shows AuthService running on a particular server and the Database on another. It's mapping logical components to physical hardware. If multiple components run on one node (like Auth API and Domain were separate logical components but both are part of the same deployed service), you might see them collectively as one artifact or multiple artifacts on the same node. - In essence, interpreting the deployment diagram gives you the **operations perspective**: you understand the structure needed to run the system in real life – what machines/instances are needed, how they connect, and where each part of the software resides ⁵⁴. This is crucial for tasks like environment setup, troubleshooting (knowing which server to check), scaling decisions, etc.

By examining the deployment diagram for the auth service, all team members (developers, DevOps, network engineers) gain a shared understanding of the infrastructure. It ensures, for example, that everyone knows the web app and database are separate (so latency might be a factor), or that the system requires certain ports open between nodes. It transforms the abstract architecture into a concrete setup blueprint.

6. Entity-Relationship Diagram (ERD)

Purpose

An Entity-Relationship Diagram (ERD) models the **data schema** of the system – it shows the database tables (entities) and the relationships between them. While similar to a class diagram, ERDs focus purely on how data is structured and related in a **relational database or data store**. The purpose is to design and communicate the **database design** for the authentication service: what tables exist, what columns (attributes) they have, which columns are keys, and how tables link via foreign keys. This is crucial for systems using SQL (like our service with SQL Server via Dapper) to ensure the database supports the domain's needs (for example, storing users, roles, permissions, etc., with proper constraints). An ERD helps database designers and developers optimize queries, ensure data integrity (through relationships and cardinalities), and align the domain model with the physical storage. It typically omits behavior and just focuses on data structure, often using notations like Crow's Foot to indicate one-to-many relationships. In short, the ERD is the blueprint for the **persistent data** in the auth service ⁵⁵.

Structure

Key elements of an ERD include:

- **Entities (Tables):** Represented as rectangles, each corresponds to a database table. The entity's name is usually at the top. Inside, attributes (columns) may be listed. Often, the primary key is listed first (and underlined to indicate it's a key) ⁵⁶, and foreign keys might be noted (italicized in some notations) ⁵⁶. In Crow's Foot style, attributes might not be listed at all in the diagram – just the entity name, to keep it focused on relationships. But details can be included if desired.
- **Attributes (Columns):** If detailed, each attribute can be listed with its name (and sometimes data type). The primary key attribute is typically **underlined** ⁵⁷. Foreign key attributes might be *italicized* or noted with (FK). Derived or calculated attributes (if shown) might be in brackets or with a different notation (dashed outline, etc.) ⁵⁸ – though these are less common in a straightforward ERD.
- **Relationships:** Lines connecting entities show how tables relate. A Crow's Foot notation is common: it uses symbols at line ends to indicate cardinality (like a “crow's foot” fork for “many”, a single line for “one”, a circle for zero possibility, etc.) ⁵⁹. For example, a one-to-many relationship from User to Session would have a single line (or '1') near User and a crow's foot (with maybe a 0.. or 1.. annotation) near Session ⁵⁹. Relationships can be labeled (optionally) with verbs or names, but often the meaning is clear by context (like “User —< has —< Session” might be implied).
- **Multiplicity/Cardinality:** This explicitly states the nature of the relationship:
 - One-to-One (1:1): A single entity instance relates to one of another. E.g., if each user had exactly one profile row (not in this domain likely, but as concept).
 - One-to-Many (1:N): One instance of A can relate to many of B, but B relates to only one of A. E.g., one *User* can have many *Sessions*, but each Session is for exactly one User.
 - Many-to-Many (M:N): Instances of A can relate to many of B and vice versa. These are usually resolved via a join table. For example, *User* and *Role* is many-to-many: a user can have multiple roles, and a role can be assigned to multiple users ⁶⁰. The ERD would show User and Role without a direct crow's foot both ways (since crow's foot is typically one side many, one side one). Instead, an associative entity (join table) is introduced.
- **Associative Entities (Join Tables):** To implement many-to-many, an extra entity is used (often just a table with foreign keys to both sides). In the ERD, this might be shown as a relationship entity (sometimes a diamond shape in Chen notation, or just a normal table in Crow's Foot style with two FKs). For instance, you might have a table `UserRole` linking Users and Roles. This would appear on the ERD connecting to both User and Role (each UserRole row belongs to one User and one Role). The diagram might denote it with a special symbol or just as another entity.
- **Weak Entities:** In Chen notation, a weak entity (one that cannot exist without a parent, lacking its own unique key) is drawn with a double border. In

Crow's Foot, you might simply see a relationship where a child's PK includes a FK from parent (like a composite key). For example, if *Session* did not have its own surrogate key and instead used (*UserID*, *SessionID*) composite, it could be considered a weak entity dependent on *User*. But often we use surrogate keys, so not needed. - **Notation Styles:** Chen notation uses diamonds for relationships and is more descriptive. Crow's Foot (which is common in many tools) is more concise and uses the crow's foot symbols. In our explanation, we lean on Crow's Foot style (as it's widely used and likely available in Draw.io's Entity-Relation shapes).

Applying to our domain model's data design, we expect tables such as: - **User** – with columns: *UserID* (PK), *Username*, *PasswordHash*, *Email*, etc., plus maybe metadata like *CreatedDate*. - **Role** – with *RoleID* (PK), *Name*, *Description*. - **Permission** – with *PermissionID* (PK), *Name*, etc., if we store individual permissions. In a simple design, you might just have *Roles* and a *RolePermissions* join table. - **UserRole** – a join table linking *Users* and *Roles*. It would have *UserID* (FK to *User*) and *RoleID* (FK to *Role*), with a composite PK on (*UserID*, *RoleID*) or its own surrogate key. This table represents the many-to-many assignment ⁶⁰. - **RolePermission** – if using permissions, a join between *Role* and *Permission* similarly (*RoleID*, *PermissionID*). - **Group** – *GroupID*, *Name*, etc. - **UserGroup** – a join table for many-to-many between *User* and *Group*. - **GroupRole** – possibly, if groups can have roles, a join between *Group* and *Role*. - **Session** – with *SessionID* (PK), *UserID* (FK), *RefreshToken*, *ExpiryTime*, etc. It's related many-to-one with *User* (*User* 1 -> * *n* *Sessions*). - **AuditLog** – with *LogID* (PK), *UserID* (FK maybe), *Action*, *Timestamp*, etc. (Could also have a composite key or just PK). - Possibly other tables like **Token** if refresh tokens are stored separately, but likely integrated in *Session*.

The ERD will display these tables and the foreign key relationships among them.

How to Build in Draw.io

1. **Enable Entity Relation shapes:** Draw.io has a dedicated "Entity Relation" shape library ⁶¹. Go to **More Shapes**, under the Software category enable "Entity Relation" and click Apply. This gives you Crow's Foot connectors and entity shapes.
2. **Add Entities (Tables):** Drag an *Entity* shape onto the diagram for each table. You might use the rectangle with multiple rows shape (some ERD shapes let you list fields). For simplicity, you could use one of the draw.io ERD styles: either a plain box for each table name (with attributes listed as text inside), or the Chen style with attributes as separate ovals (less common in simple guides). The second style in draw.io (as mentioned ⁶²) has attributes listed in the box, which is easier to read for relational design.
3. **List Attributes (optional):** If the shapes allow editing text inside: Type out the primary key first (maybe prefix with **PK** or underline it by Ctrl+U if you want), then other columns. Mark foreign key columns with **FK** or italic font. For example, for **UserRole** table, list **PK, FK UserID**, **PK, FK RoleID**. If not listing attributes, you can represent keys graphically: some notations use a key icon or just trust the relationship lines to imply keys. But listing at least key fields is helpful.
4. **Use Crow's Foot Connectors:** In the Entity Relation library, select the appropriate connector. For instance, choose the connector that has a crow's foot on one end and a line on the other (that's for one-to-many). Drag it from the child table to the parent table. For example, drag crow's foot from *Session* table and connect to *User* table, so that at *Session*'s end we see the fork (many sessions) and at *User*'s end a single line (one user). Draw.io might allow you to attach these connectors to the entity shapes so they stay connected when moved.

5. For many-to-many, you need an intermediate table. Connect one-to-many from the join table to each of the main tables. For instance, UserRole -> User (many userroles per one user) and UserRole -> Role (many per one role). This effectively models the M:N.
6. **Cardinality Labels:** If not using crow's foot notation explicitly, you can label min/max near the line ends. For example, next to User at the Session relationship, you might put "1" and next to Session "0..*" to indicate a user can have zero to many sessions ⁶⁰. Often crow's foot shapes include some of this (like a circle at one end for zero possibility).
7. **Adjust Layout:** Arrange tables so lines don't overlap heavily. A common approach: put central tables like User in the middle, and related tables around it. E.g., User in center, Session below it, AuditLog below, UserRole to the side, connecting to Role further out, etc. Group join tables in between their parents (UserRole in between User and Role, to make the relationships clear).
8. **Notation Legend (optional):** If your audience isn't familiar with crow's foot symbols, you might add a small legend (e.g., a sample of the notation meaning 1, 0..*, etc.). But generally, crow's foot is widely understood in ERDs.
9. **Validate Keys:** Ensure every relationship line corresponds to an actual foreign key. E.g., Session to User implies Session table has a UserID FK. The diagram should allow someone to see, for each table, which other table it references or is referenced by.

Interpretation

When you look at an ERD for the auth service, you're reading the **database schema design**: - Each **entity (table)** name is a collection of similar data. For example, a box labeled "User" is the user table. If attributes are listed, you can see what fields it contains. The primary key (often underlined or with PK label) indicates the unique identifier for that table - e.g., `UserID` as PK of User. That's crucial to see how other tables link to it. - **Relationships (lines)** show foreign key links: - If you see a line connecting Session to User with a crow's foot near Session, interpret that as *User (1) to Session (many)* ⁶⁰. In practical terms, the Session table has a foreign key `UserID` referencing the User table's primary key. Thus, one user can have multiple session records. - A crow's foot at both ends would normally imply many-to-many, but usually, it won't be directly between two main tables. Instead, check if there's a join table. For example, if an ERD shows a **UserRole** table in between with one-to-many from UserRole to User and one-to-many from UserRole to Role, that indicates a many-to-many between User and Role (UserRole is the associative entity). So you interpret: a user can have many roles (through many UserRole entries), and each role can belong to many users (through many UserRole entries) ⁶⁰. - The notation might also explicitly mark these as PK, FK. So in the UserRole entity, "UserID, RoleID (PK)(FK)" means those are both part of primary key and foreign keys to respective tables. If drawn as a diamond (Chen style) between User and Role, that diamond would label the relationship like "assigns" or something, but crow's foot approach with a join table named is clearer in a relational sense. - **Optionality:** If a relationship has a circle (0) at one end, it means that side can have zero instances. For example, if a user could possibly have no roles, the crow's foot near UserRole might start with a circle (0..). *Often, by default, we assume 0.. for the "many" side and 1 for the "one" side, unless not null is enforced.* In an auth system, likely every session must have a user (so it's one user per session required, the user side is mandatory, which is normally the case with FK constraints unless you allow nulls). We probably assume mandatory relationships by default (like a Session cannot exist without a User). - **Key propagation:** Look at each table to see what foreign keys it contains: - Session has a UserID FK to User (so you know how to join Session to User in queries). - AuditLog likely has UserID FK (so you can attribute actions to users). - UserRole has UserID FK to User and RoleID FK to Role, meaning you join through it to connect users and roles. - RolePermission (if present) connects Role and Permission similarly. - If Group and Group membership tables exist, similar logic: GroupUser join table links Group and User. - **Hierarchies or dependencies:** If a table is connected via identifying relationship (composite PK including parent's PK), it

might be drawn differently (but in our domain, not likely needed except join tables which effectively use composite PKs). - **Cross-check domain vs DB:** This ERD should mirror the domain model but in a relational manner. For example, the class diagram might have had a User–Role association many-to-many; the ERD concretely shows UserRole table. The class diagram had a User–Session one-to-many; the ERD shows that too. If something is in the domain but not in the ERD, it means either it's not stored (like perhaps JWT tokens themselves might not be stored except refresh tokens in Session). - Because Dapper is used (micro-ORM), the schema's clarity is important for writing queries. As a reader, if I see these tables, I can imagine the SQL queries: e.g., to get a user's roles, join User->UserRole->Role. To validate a refresh token, look in Session table where Token matches and check User. The ERD thus directly informs how queries or Dapper mappings will be set up. - **Data integrity rules:** If any cascading or special constraints are to be noted, sometimes an ERD will mark them (like a double line for identifying relationships). But likely, the main thing is to see where the foreign keys are. For example, if a user is deleted, ideally their sessions and userroles should be deleted too (either by cascading or by application logic) – the ERD doesn't show the cascade, but the existence of relationships hints at that need. - Another detail: Primary keys and unique constraints – the ERD shows primary keys (underlined). If there were any unique constraints outside PK (like email must be unique in User), the diagram might not explicitly show it unless annotated. But sometimes an asterisk or so can denote unique. Usually not shown unless it's a key attribute. - To interpret cardinality in plain terms: for each relationship, make a sentence: - “A User can have many Sessions; each Session belongs to exactly one User.” - “A User can have many AuditLogs; each AuditLog is for one User.” - “A User can have many Roles *through* the UserRole table, and a Role can be assigned to many Users.” - “A Role can include many Permissions, and a Permission can be in many Roles (hence a join table RolePermission).” - “A User can belong to many Groups, a Group has many Users (join table UserGroup).” - These align with domain concepts of the auth system. - If any entity stands alone (no relations), that's rare in such a domain – perhaps `Permission` might only connect to Role. But presumably all should be connected except maybe if some audit doesn't link to user (but usually it does). - Summarily, reading the ERD gives you the **database perspective** of the system's identity and access data. You ensure the relationships match what's needed for queries (e.g., to fetch permissions of a user, you'd join through roles, which is possible thanks to the relationships drawn). It also reveals if any data is not captured: e.g., do we store token issuance and expiry? Yes, in Session (with refresh token and expiry). Do we store password history? Not indicated, so presumably not, etc.

Thus, the ERD for the authentication service confirms that the persistence layer is well-designed to support user authentication, authorization (via roles/permissions), grouping, session management, and auditing in a normalized manner ⁵⁵. It ensures that developers and DB admins share an understanding of how data is structured and how to enforce referential integrity among the auth-related data.

7. System Architecture Diagram

Purpose

A system architecture diagram provides a **holistic, end-to-end view of the entire system's structure**, often blending both software and hardware elements to show how the system's components, external systems, and users all fit together ⁶³. It's somewhat similar to a high-level component diagram or a Level 1 C4 context diagram – but it isn't limited to UML notation. The goal is to capture *everything in one picture*: the users, the client applications, the services (like our auth service), other dependent or interacting systems, data stores, and their interactions. This diagram is great for communicating with a broad audience because it abstracts the technical details into a clear map of the system and its environment. For our DDD-based

authentication service, the system architecture diagram would illustrate how the authentication service integrates into the larger ecosystem (for example, how front-end or other microservices use it), and how requests flow from the very front (user/UI) through to the backend and database. Essentially, it's the "big picture" showing **all tiers** – often combining elements of the deployment (nodes), component (software modules), and context (actors) diagrams into one cohesive overview ⁶⁴ ⁶⁵ .

Structure

The structure is not strictly standardized (unlike UML diagrams). It typically includes:

- **Users/Actors:** Graphical representation of end-users or external systems that initiate interactions. Could use persona icons or stick figures labeled "End User" or "Admin User", etc. Possibly also external systems if, say, an external service calls our auth API (maybe not in our case, unless other microservices are considered as actors).
- **Client Application Layer:** If the system includes a client app or UI, that can be shown (like a web browser icon or mobile app icon) interacting with the user and sending requests to the server. This is the front-end of the flow.
- **System Components/Services:** The major subsystems or services of our system. For the auth service, that would be depicted (perhaps as a box labeled "Authentication Service"). If the auth service is part of a suite of services, you might show an "API Gateway" or separate front-end service that communicates with it. Internally, one might break the auth service into layers (presentation, domain, DB) but often the system arch diagram might just keep it as one box for simplicity, or a few boxes to show internal modules if needed.
- **Databases/Storage:** Show the databases (SQL Server) or other storage as separate elements (like a cylinder icon for a database). In our case, the Auth database.
- **External Integrations:** If the auth service relies on or talks to external systems (like an email server for sending verification emails, or an external identity provider, etc.), those would be shown as well.
- **Connections/Arrows:** Arrows or lines labeled with interaction or data flow. For example, an arrow from the user's browser to the Auth Service labeled "HTTPS Request (Login)", then an arrow from Auth Service to Database labeled "SQL queries". Possibly arrows from Auth Service to other services if tokens are validated or user data is fetched, etc. Or an arrow from other microservices to Auth Service labeled "Token Validation call" (though with JWT, usually no need to call auth service for validation, but if it were OAuth introspection, it might).
- **Boundary/Groupings:** You might group components into tiers or contexts – e.g., a box encompassing the Auth Service and DB labeled "Authentication Service (bounded context)" or grouping front-end vs back-end. This can illustrate trust boundaries or contexts.
- **Notation:** Many system architecture diagrams are drawn informally or with simple icons (clouds, servers, user icons) rather than strict UML. The emphasis is clarity. For example, using a cloud symbol to represent the internet or a corporate network, with arrows passing through it.
- **DDD context note:** Since our system is DDD-based, we might indicate the "Auth Domain" as a bounded context. If we wanted to highlight that, we could label the Auth Service as belonging to the "Identity and Access Management Context" in a larger system. If the diagram covers multiple bounded contexts, we'd draw each and show relationships. But presumably, this guide centers on the auth context mainly.

In summary, the system architecture diagram for the auth service might show:

- The **User** (actor) using a **Client App** (like a web UI).
- The client communicates with the **Auth Service (API)** over HTTPS.
- The Auth Service, as part of its architecture, communicates with the **Auth Database** (SQL) for storing user credentials, sessions, etc.
- The Auth Service issues **JWT tokens** back to the client.
- The client then uses those JWT tokens to call other **Application Services or APIs** (if we include them, e.g., an Orders Service, a Products Service in an e-commerce, etc.). Those other services validate the token (either internally or by using a shared public key).
- Optionally, if there's an **Email Service** (for sending confirmations) or external IDP, they'd be included.

This gives an end-to-end trace: user -> auth -> token -> use token for other parts.

How to Build in Draw.io

1. **Choose an appropriate shape set:** For a general architecture diagram, you might mix and match shapes. Enable **General**, **UML**, and possibly **Networking** or **Misc** shape libraries. Draw.io has many icons (you can search the shape library for keywords like "user", "server", "cloud"). You could also use simple rectangles with text and small icons in them for clarity.
2. **Place Actors/Clients:** Drag a user icon for the end-user at the far left (for example). Label it "User". If there's a client application, maybe put a browser or mobile phone icon right next to the user to imply the user through that app interacts. Label that "Client App (Browser)".
3. **Place the Auth Service:** Draw a box or server icon more to the right. Label it "Authentication Service (.NET 10)" for instance. Inside or below it, you might list its key sub-components (like "Auth API, Auth Domain, Dapper Repo"), but keep it simple. A server icon or a stack icon could represent the service.
4. **Place the Database:** Use a cylinder icon for the database to the bottom-right of the Auth Service. Label it "Auth DB (SQL Server)".
5. **Draw other systems if needed:** For context, if you want to show that after login the user accesses other services: draw additional boxes (e.g., "Protected Resource Service" or some other API) to the right of Auth Service. Or an arrow directly from User to "Other Services" with note "subsequent API calls with JWT". Alternatively, show just one generic "Other Microservice" and a note.
6. **Connect with Arrows:** Draw an arrow from User (or Client) to Auth Service labeled "Login Request (credentials)". Then an arrow back from Auth Service to User labeled "JWT Token/Response". Draw an arrow from Auth Service to the Database labeled "Read/Write User data (via Dapper/SQL)". If including the after-login scenario: arrow from User to Other Service labeled "API call with JWT", and maybe that service validating token internally or via Auth – you can either show that service verifying the JWT signature itself (maybe a note "JWT validation via public key" on the arrow) or, if not to complicate, one can omit.
7. **Optional additions:** If the auth service communicates with an email/SMS service (for OTPs or confirmation emails), include an "Email Service" icon and an arrow from Auth Service to it ("Send verification email"). If an external OAuth provider is used for social login, include those logos or icons too, connecting to Auth Service as needed.
8. **Visual groupings:** Possibly draw a cloud around the Auth Service and DB to indicate the server side (like "Backend Cloud"), and the user and client outside it. Or label the network: an "Internet" cloud between client and the service if it's over internet. Also you might annotate "DMZ" or firewall if showing a security boundary.
9. **Legend:** Usually not needed if icons are self-explanatory, but ensure labels are clear. Use color if it helps differentiate (maybe all backend components in one color).
10. **Keep it simple:** The system arch diagram should be easily digestible in seconds. So avoid too many technical notations; prefer descriptive labels. For instance, rather than showing an internal application layer, just labeling the box "Auth Service (includes API, domain logic)" suffices.

Interpretation

When viewing the system architecture diagram: - Start with the **users/actors**: Recognize who initiates interactions. If you see a user figure and a browser, you understand there's a human user operating via a client app. For example, "User on Browser" indicates the front-end is a web app. - Follow the **arrows** depicting user requests. An arrow from the browser to "Auth Service" labeled "Login API Call" shows that the first step is the user sending credentials to the authentication service endpoint. This is typically an HTTP call

over the internet (perhaps denoted by an internet cloud icon or “HTTPS” on the arrow). - The **Auth Service** component in the diagram is central. Note what’s said about it (maybe a subtitle “.NET Core Service with DDD” etc., or it might implicitly contain multiple layers). It likely has arrows going out to other components: one downwards to the database, meaning it queries/stores user and session data there. Possibly one to an email system if drawn. Interpret these as: the Auth Service is responsible for interacting with persistent storage and other necessary subsystems to fulfill authentication. - The arrow from Auth Service back to the user (browser) labeled “JWT Token” implies the output of a successful login: the system issues a JWT. So the user receives a token. This indicates stateless auth – the token will be used subsequently. - If the diagram shows subsequent interactions: e.g., an arrow from user’s browser to “Other API Service” with a note “[Token in Authorization header]”, that indicates how the token is used to authenticate to other parts of the system. If an “Other API” box is present, it demonstrates that the Auth Service is part of a broader system where other services rely on the tokens it issues for user identity ⁶⁶. It might also show those other services verifying the token (if drawn, maybe a note or an arrow from Other Service to Auth Service if using an introspection endpoint – but with JWT usually they verify locally). If it’s not shown, one assumes JWT validation is done internally by those services using the token’s signature. - The **database** cylinder under Auth Service shows that the service has its own database (likely containing user accounts, roles, etc.). If labeled SQL Server or something, that confirms technology. The arrow from Auth Service to DB likely is labeled “SQL queries” or similar, meaning the service reads/writes user info (like checking password, saving sessions) in that DB. - If multiple user roles or other actors are drawn (like an Admin user who uses an Admin UI to manage roles), look at their interactions. Maybe an Admin UI also calls the Auth Service’s admin endpoints (like create user, assign role). If that’s drawn, it highlights admin functionality. - Check for any **external systems** or third-party integration: For instance, if the diagram had a box “External Identity Provider (OAuth2)” with an arrow to Auth Service (perhaps for social logins), you’d interpret that the Auth Service interacts with that provider (maybe to validate tokens from Google, etc.). If an email/SMS service is shown, you’d interpret the Auth Service sends out-of-band messages via those. - **Trust and boundaries:** Are there any dashed lines or different background shading indicating a trust boundary? Maybe the diagram isolates the “Auth Service & DB” within a “Secure Backend Network” and the user outside. If an API Gateway is shown at the boundary, that means the user’s requests first hit the gateway then routed to Auth Service. Understanding these boundaries is key for security architecture. For example, an API Gateway could throttle or route requests, which adds an extra component in flow. - Because the question specifically mentions DDD and full end-to-end, the system arch diagram might also illustrate the layering inside Auth Service: e.g., it might show something like a layered stack: **Client -> API (Interfaces layer) -> Domain layer -> Infrastructure (DB)** to emphasize DDD layers. If so, it might be drawn as vertical layers within the Auth Service box. Interpretation would be that within the Auth Service, the presentation (API controllers) calls the application/domain layer which then uses infrastructure (Dapper/DB). That is more an internal view but could be shown. Recognizing that means the system is designed with separation of concerns (as expected in DDD). - The overall view should answer: *Who uses the system? Through what interface? What happens internally? What other parts of the system or environment are involved?* For example: the end-user (actor) interacts via a browser (UI) with the Authentication Service (the main subject here). The Auth Service handles authentication by checking data in the database and then responds with tokens. Those tokens allow access to other system components (like other microservices), which validate them. The Auth Service and database are deployed on certain servers (which might also be indicated if this diagram merges deployment info – e.g., maybe a mention of “hosted on Azure Cloud” or similar). - The difference between this and previous diagrams: it’s less formal than the UML ones, combining various aspects. But it gives a **connected narrative** of the architecture. One can trace a full user story: *User opens app -> sends login to Auth -> Auth verifies with DB -> Auth returns token -> User calls other APIs with token -> those APIs trust the token.* This matches what one expects from a JWT-based auth system ³². - The system architecture diagram is often the one you’d show to an architect or new developer first to orient them, before diving into, say, the

class or sequence diagrams. It's essentially a **high-level map**. Interpreting it quickly gives you context for all the other diagrams. For example, it shows that domain logic resides in a service that also has a DB – which is consistent with the deployment diagram's info but in a more functional context.

In essence, by studying the system architecture diagram, you understand **how the authentication service fits in the world**: who interacts with it, what other systems or layers it involves, and how data and control flow from the user's action all the way through to data storage and back. It ensures you see the end-to-end pipeline from **frontend to domain logic to persistence to other systems**, exactly as requested in the scope, but in one integrated diagram ⁶⁴. This holistic perspective is invaluable for high-level discussions and for verifying that the overall architecture meets requirements (e.g., security, scalability, integration points) before zooming in on specifics.

8. Activity/Flowchart Diagram

Purpose

An activity diagram (or flowchart) illustrates the **workflow or process logic** step-by-step, including decisions, parallel processes, and loops ⁶⁷. It is a **behavioral diagram** like sequence diagrams, but rather than focusing on message exchanges between objects, it focuses on the sequence of activities and the conditions for branching. The purpose is to capture the **business logic or algorithm** in a visual form. In our authentication service, an activity diagram could be used to describe the login process flow (from entering credentials to obtaining a token, including decision points for valid/invalid credentials), or the password reset flow, or token refresh flow. It's useful for understanding and communicating how a particular use case or complex operation *flows*, especially if there are multiple decision branches (e.g., invalid password vs account locked vs success). It can also show concurrent activities (though in an auth context, concurrency might be less relevant except maybe multi-factor flows or asynchronous logging). Essentially, it's like a **flowchart** that can be used by developers or even non-technical stakeholders to verify that the logic covers all cases and correct order (e.g., "check credentials -> if ok then issue token else error" is clearly shown) ⁶⁸.

Structure

Activity diagrams comprise:

- **Start Node**: A filled circle symbol indicating the start of the workflow.
- **Activities (Actions)**: Rounded rectangles (or sometimes just rectangles with smooth edges) representing each step or action in the process. For example, "Receive Login Request", "Validate Credentials", "Generate JWT", "Save Session to DB", "Return Response".
- **Decision Nodes**: Diamonds that represent branching. A question or condition is evaluated, leading to different outgoing arrows for different outcomes (often labeled [condition] or [yes]/[no]). For instance, after "Validate Credentials", a diamond with "Credentials valid?" might branch to two paths: one labeled [Yes] and one [No].
- **Merge Nodes**: Diamonds can also serve to merge branches back together after an if/else.
- **Fork/Join (Parallel) Nodes**: If some activities can happen in parallel, a thick horizontal or vertical bar is used to split (fork) and later join parallel flows. In an auth scenario, parallel flows are less common, but maybe logging and notification could be parallel to main response. Generally, in our context, likely sequential.
- **End Node**: A bullseye (filled circle with a surrounding outline) indicating the end of the process.
- **Flow arrows**: Connectors with arrows show the direction of flow from one step to the next. They often go from an activity to a decision or to another activity. They can be labeled if not obvious (like labeling branches out of a decision with guard conditions).
- **Swimlanes (Partitions)**: Optionally, activities can be organized into swimlanes each corresponding to an actor or component responsible for those actions. For instance, one lane for "Client", one for "Auth Service", one for

“Database” could separate actions done by each. This can merge some sequence diagram info into the activity diagram. If we use swimlanes, then “Enter Credentials” might be under Client, “Verify Password” under Auth Service, “Query DB” under Database lane, etc., showing responsibility. - **Objects/Artifacts:** Sometimes UML activity diagrams show objects being produced or used at certain points (like an object node). E.g., after “Generate Token”, an object “JWT Token” might appear as output. But this is optional and often not needed if the flow is clear.

For a **JWT authentication flow** example, an activity diagram might look like: - **Start** → *User submits login form* (this could be an action in a Client swimlane) → *Auth Service receives credentials* → *Hash provided password and retrieve stored hash for user* → *Compare hashes* (this is a decision: valid?) ⁶⁸ : - If **no** (password mismatch): → *Increment failed login counter*, maybe another decision “lockout reached?” (if yes, *lock account*) → then *Return error (401 Unauthorized)* → **End**. - If **yes** (valid credentials): → *Generate JWT access token and Generate Refresh token* → *Store refresh token in DB (Session record)* → *Return tokens in response* → **End**. - That covers successful and failure paths. The diagram might also include an action for *Audit log the login attempt* either in each branch or a merged action after decision (maybe better to log both success and failure attempts). - Possibly, if multi-factor auth was in scope, the diagram could branch into a subflow for “MFA required?” etc.

Another example could be the **Refresh Token flow**: - Start when user calls refresh endpoint → check if provided refresh token is valid (exists, not expired) → if valid, issue new access token (and maybe new refresh token, update DB) → else, deny and force re-login → end.

But likely one diagram (the login process) is enough for demonstration in the guide.

How to Build in Draw.io

1. **Enable UML (Activity) shapes:** Draw.io's General or Flowchart shape library has basic flowchart symbols (start, decision, etc.). The UML shape library also includes activity symbols, but using flowchart shapes is fine. Enable “Flowcharts” and/or “UML” shapes.
2. **Create Swimlanes (optional):** If using swimlanes, one approach: use the container shape “Swimlane” from flowcharts or manually draw rectangles to divide lanes. Label them “Client”, “Auth Service”, “DB” etc. If not using lanes, skip.
3. **Place Start node:** Drag a filled circle for start.
4. **Add Action shapes:** Drag rounded rectangles for each action step. Order them roughly top to bottom. E.g., “Receive Login Request”, “Lookup User in DB”, “Validate Password”, “Create Tokens”, “Write Session to DB”, “Send Response”.
5. **Add Decision diamonds:** Drag a diamond for conditions like “Credentials valid?”. Label the outgoing arrows: one arrow from diamond to success path actions (label “[Valid]” or simply put the condition text on the line), another arrow to failure path (label “[Invalid]”).
6. **Continue flow:** Connect shapes with arrows in the logical order. Ensure the arrowheads point correctly. In draw.io, you might need to use line connectors. Label arrows if the condition or trigger isn't obvious.
7. **End node:** Drag the bulls-eye end symbol where flows terminate. You may have multiple end points for different outcomes or sometimes you merge back to a single end.
8. **Fork if needed:** If you want to show concurrent tasks (for example, maybe simultaneously log the login attempt while generating token – though in code that would still be sequential unless threaded), you can use a fork bar. But likely not needed here.

9. **Styling and readability:** Align and space elements so that the "happy path" (successful login) flows straight down one side, and alternate path branches off to the side then either loops back or ends. Use color to highlight success vs failure maybe. Put the diamond after an action "Validate Password" to branch into two flows.
10. **Add any text annotations:** If certain steps need clarifying notes (like "Password hashing uses Bcrypt"), you can attach a note shape near that action.

Interpretation

To interpret an activity/flowchart diagram: - **Follow from Start to End:** Begin at the solid circle (start) and follow the arrows. This traces the sequence of actions. - At each **Activity box**, understand that as a step being performed. For example, "Validate Credentials" means the system is checking the username/password against what's stored. - When you hit a **Decision diamond**, read the condition and see the branching arrows. Each outgoing arrow will have a label or an implicit meaning. E.g., a diamond labeled "Password correct?" might have one branch (arrow) with [Yes] going to "Generate Tokens" and another with [No] going to "Return Error". You interpret that as an if/else: if password is correct, do the yes branch; if not, do the no branch ⁶⁸. - If swimlanes are present: note which column the action is in. That tells you *who/what* performs that action. For instance, an action in the "Client" lane like "Enter Credentials" means the user on client side is doing it. The next action "Verify user exists" in "Auth Service" lane means the server does that by querying DB (the next action might actually be in DB lane "Retrieve user record"). This division of responsibilities is similar to a sequence diagram but in a more flow-oriented way. It helps ensure that, for example, a certain step isn't mistakenly assigned to the wrong party (client vs server). - Look for **parallel bars** if present. If a fork bar splits into two arrows leading to two sets of activities, that means those actions happen concurrently. A join bar later will synchronize them. In login, it's probably linear, but if you see any, interpret accordingly (maybe something like "While generating token, in parallel log event" – which likely isn't drawn but just hypothetically). - **End points:** See where the flows terminate. There might be an end for success and a separate end for failure, or they might merge back into a single end. For example, a failure path might go directly to an end marked "Authentication Failed (End)" whereas success path goes to "Tokens issued (End)". This indicates the outcome of each branch. If they merge (like after both success and failure they go to a common end node or maybe loop back to start if considering a continuous process), interpret that carefully. Usually login would just end with result. - Pay attention to any **loops**: depicted either by a backward arrow or using a loop stereotype. For instance, if the diagram had a loop for "User retries password up to 3 times", you'd see a flow returning from a failure branch back to an earlier step (with maybe a counter). Not explicitly needed in basic flows unless specified. - Also note if any **object flows** were shown: maybe not in our likely diagram, but if an object "JWT Token" is shown being output from an action and used in another, that indicates data artifact moving. Eg: "Generate JWT" outputs a [JWT] object that flows to "Return Response" action. It's fairly intuitive if present. - The activity diagram is essentially a logical flowchart: Each step leads to the next in a cause-and-effect manner. Interpreting it is often straightforward ("first do A, then check B, if B is true do C, otherwise do D..."). Ensure no branch is left unresolved (each decision has matching outcomes drawn, etc.). The diagram should cover all possibilities of that process. - In the context of the auth service, interpreting the login flow activity diagram ensures understanding of the **logic and order**: e.g., it makes clear that the system *always* verifies credentials before generating tokens, and that it handles the invalid case by returning an error without attempting token generation or DB updates beyond maybe logging the attempt ⁶⁹. It might also highlight secondary steps like updating last login time or audit logging if included as actions. - Compare it mentally to the sequence diagram: They should align. The sequence diagram might show which components interact, while the activity focuses on the *decision making and sequence of operations*. For example, sequence shows how AuthService calls DB then returns token, and activity shows within AuthService, after DB returns user data, a decision "is password

correct?" happens, etc. They are two views of similar scenarios – one object-centric, one process-centric ⁷⁰ .

- For a non-technical stakeholder, the activity diagram (especially if drawn without swimlanes) is easier to follow than sequence. It reads like a flowchart anyone can follow for the login process. It's essentially illustrating the algorithm the auth service uses. So ensure to glean from it: the *business rule* that wrong password leads to error vs right leads to success is clearly visible, any additional rules (like account locked after 5 fails) would be another decision branch if included.
- All in all, reading the activity diagram gives you a clear picture of *the logic flow for a use case*, which complements the static and structural understanding from earlier diagrams ⁷¹ . It's a way to verify that the implementation steps (or planned steps) cover all paths, and to communicate that logic in a straightforward way.

By combining the insights from each type of diagram above, one gains a **comprehensive understanding** of the authentication service's design: the *requirements and actors* (Use Case), the *static domain structure* (Class/ERD), the *dynamic behaviors and interactions* (Sequence/Activity), the *modular software organization* (Component/System Architecture), and the *physical deployment* (Deployment). Each diagram serves a different but complementary purpose in the documentation and design process, and following best practices for constructing them (as outlined with Draw.io tips) ensures they are clear and useful for the intended audience.

¹ ⁶ ⁷ ²¹ ³⁷ ⁴⁶ ⁴⁹ ⁵⁴ ⁶⁷ ⁶⁸ ⁷¹ Unified Modeling Language (UML) Diagrams - GeeksforGeeks

<https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/>

² ³ ⁴ ⁵ ⁷⁰ UML use case diagrams with draw.io - draw.io

<https://drawio-app.com/blog/uml-use-case-diagrams-with-draw-io/>

⁸ ⁹ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ UML class diagrams in draw.io - draw.io

<https://drawio-app.com/blog/uml-class-diagrams-in-draw-io/>

¹⁰ Using Domain-Driven Design to to Create Microservice App | by Eugene Zimin | Medium

<https://medium.com/@eugene-zimin/using-domain-driven-design-to-to-create-microservice-app-e234154a3fd1>

¹¹ ¹² ¹⁸ ¹⁹ ²⁰ Best Practice - An Introduction To Domain-Driven Design | Microsoft Learn

<https://learn.microsoft.com/en-us/archive/msdn-magazine/2009/february/best-practice-an-introduction-to-domain-driven-design>

²² ²³ ²⁴ ²⁵ ²⁶ ³⁴ ³⁵ Blog - Create a sequence diagram

<https://www.drawio.com/blog/sequence-diagrams>

²⁷ ²⁸ ²⁹ ³⁰ ³¹ ³² ³³ ³⁶ ⁶⁶ ⁶⁹ Architecture Patterns in Microservices: Security with JWT - Paradigma

<https://en.paradigmigital.com/dev/architecture-patterns-microservices-security-jwt/>

³⁸ ³⁹ ⁴⁰ ⁴¹ ⁴² ⁴³ ⁴⁴ ⁴⁵ ⁴⁷ ⁴⁸ Blog - UML component diagrams show the structure of a system

<https://www.drawio.com/blog/uml-component-diagrams>

⁵⁰ ⁵¹ ⁵² ⁵³ Create UML deployment diagrams in draw.io - draw.io

<https://drawio-app.com/blog/create-uml-deployment-diagrams-in-draw-io/>

⁵⁵ ⁵⁶ ⁵⁷ ⁵⁸ ⁵⁹ ⁶⁰ ⁶¹ ⁶² Entity Relationship Diagrams (ERDs) with draw.io - draw.io

<https://drawio-app.com/blog/entity-relationship-diagrams-with-draw-io/>

⁶³ Architectural diagrams: What is, how to draw and templates

<https://miro.com/diagramming/what-is-software-architecture-diagramming/>

